

FACULTY OF ENGINEERING
DEGREE ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING



MODERN COMPILER DESIGN (BE103701)

Faculty Name – Radhika Gokani

Chapter -6 Run-Time Environments

1. Source Language Issues

Source Language Issues

MODERN COMPILER DESIGN
(BE103701)

The source language is the language in which the program is written by the programmer. Its design and definition have a direct impact on the compiler, the efficiency of the generated code and the ease of programming.

1. What is Source Language ?

Source language (SL) is a high level language or low level language in which the source program is written. The compiler takes this program as input and translates it into target (machine) code.

2. Important Issues in Source Language Design

1. **Simplicity** - The language should be simple and easy to learn and use.
2. **Readability** - Programs written in the language should be easy to read, understand and maintain.
3. **Writability** - The language should allow programmers to express their ideas easily.
4. **Reliability** - The language should support writing reliable and error-free programs.
5. **Cost of Compilation** - The compiler for the language should be easy and economical to implement.
6. **Efficiency of Generated Code** - The language should allow the compiler to produce efficient target code.
7. **Portability** - Programs written in the language should be portable across different systems with little or no modification.
8. **Orthogonality** - The language features should be independent of each other and work uniformly.
9. **Extensibility** - The language should be designed in such a way that it can be extended to include new features.
10. **Strong Type Checking** - The language should support strong type checking to detect errors at compile time.

3. Categories of Source Languages

- **Low Level Language (LLL)**
 - Machine dependent
 - Examples : Assembly language, Machine language
- **High Level Language (HLL)**
 - Machine independent
 - Examples : C, C++, Java, Python, Pascal
- **Very High Level Language (VHLL)**
 - Closer to human language
 - Examples : SQL, MATLAB

4. Factors Affecting the Source Language

- Target Machine Architecture
- Intended Application Area
- Implementation of Compiler
- User Requirements

5. Summary Table

Issue	Description	Objective
Simplicity	Easy to learn and use	Reduce learning time and efforts
Readability	Easy to read and understand	Improve program maintenance
Writability	Easy to express ideas	Increase programmer productivity
Reliability	Supports writing correct programs	Reduce runtime errors
Cost of Compilation	Compiler easy and cheap to implement	Reduce development cost
Efficiency	Generates efficient target code	Improve execution speed and memory use
Portability	Programs run on different systems	Increase program reusability
Orthogonality	Features independent and uniform	Simplify language understanding
Extensibility	Language can be extended	Adapt to future requirements
Type Checking	Detects errors at compile time	Improve program correctness

Conclusion : A well-designed source language makes programming easier, compilation efficient and programs reliable and portable.

2. Storage Organization

Storage Organization

MODERN COMPILER DESIGN
(BE103701)

Storage organization refers to the way data such as variables, constants, temporaries, arrays, records, and procedure information are stored and managed during the compilation and execution of a program.

1. Need of Storage Organization

- Efficient use of memory.
- Quick access to data.
- Support for scopes and lifetimes of variables.
- Facilitate run-time storage management and address allocation.

2. Storage Areas in a Program

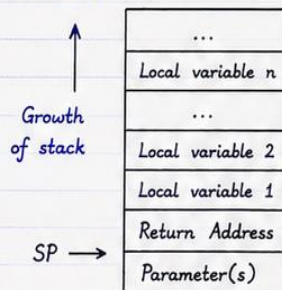
Code Area	→ Stores executable code.
Static Area	→ Stores global/static variables.
Stack Area	→ Stores local variables, parameters, return addresses.
Heap Area	→ Stores dynamically allocated memory (at run time).

3. Description of Storage Areas

Area	Purpose	Lifetime	Access	Example
Code Area	Stores compiled program instructions.	Entire program execution.	Read only.	Machine code, constants.
Static Area	Stores global and static variables.	Entire program execution.	Direct.	Global variables, static variables.
Stack Area	Stores local variables, parameters, return addresses.	During function call.	LIFO (Last In First Out).	Local variables, function calls.
Heap Area	Stores dynamic data allocated at run time.	Until explicitly deallocated.	Through pointers.	Dynamic memory (malloc, new).

4. Stack Organization

- Follows LIFO principle.
- Used for function calls, local variables, and parameters.
- Stack pointer (SP) manages the top of the stack.



5. Heap Organization

- Used for dynamic memory allocation.
- Grows during program execution.
- Managed by memory allocation functions (malloc, free / new, delete).



6. Important Points

- Efficient storage organization improves compiler performance.
- Correct management of storage areas ensures proper execution of programs.
- Overflows (stack/heap) and memory leaks should be avoided.

★ Proper storage organization is essential for efficient compilation, execution and effective memory utilization.

3. Stack Allocation of Space

Stack Allocation of Space

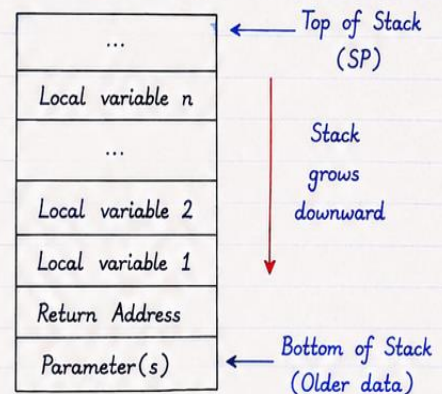
MODERN COMPILER DESIGN
(BE103701)

Stack allocation of space is a method in which memory is allocated in a Last-In-First-Out (LIFO) manner using a stack. It is mainly used to manage procedure calls, local variables, parameters and return addresses.

1. Basic Idea

- A stack is a linear data structure that grows and shrinks dynamically.
- Memory is allocated when an item is pushed on the stack (e.g., procedure call).
- Memory is deallocated when the item is popped from the stack (e.g., return from procedure).
- Thus, the most recently allocated space is the first to be released.

Stack Growth



2. Stack in Procedure Calls

- When a procedure is called, a block called Activation Record (or Stack Frame) is pushed onto the stack.
- When the procedure finishes, its activation record is popped from the stack.

3. Layout of an Activation Record

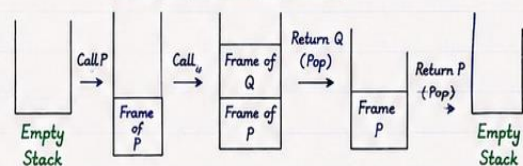
Actual Parameters	Return Address	Control Link (Saved SP/FP)	Access Link (Optional)	Local Variables	Temporary Values
-------------------	----------------	----------------------------	------------------------	-----------------	------------------

← Usually stack grows from high address to low address →

4. Example

- Consider procedure P calls Q.
- When Q is called, its activation record is pushed on the stack.
- After Q returns, its record is popped and control goes back to P.
- This continues in LIFO order.

Stack Operation (LIFO)



5. Advantages

- Simple and efficient memory management.
- Fast allocation and deallocation (just push/pop).
- Naturally supports procedure calls and recursion.
- No external fragmentation.

6. Limitations

- Limited size (stack overflow if memory exceeds).
- Only LIFO access (cannot access middle elements directly).
- Not suitable for large, long-lived data.

★ Stack allocation of space is the foundation of runtime memory management in most programming languages and is crucial for supporting procedure calls, recursion and nested block structures.

4. Access to Nonlocal Data on the Stack

Access to Nonlocal Data on the Stack

MODERN COMPILER DESIGN
(BE103701)

In many programming languages, a procedure may need to access data that is not local to it (i.e., data declared in an enclosing procedure but not passed as a parameter). Such data is called **nonlocal data**.

1. What is Nonlocal Data?

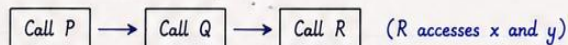
- Nonlocal data refers to variables that are declared in some procedure that lexically encloses the current procedure but are not declared within the current procedure.
- To access such data, the system must be able to locate the proper storage on the stack even though the variable is not in the current activation record.

2. Example

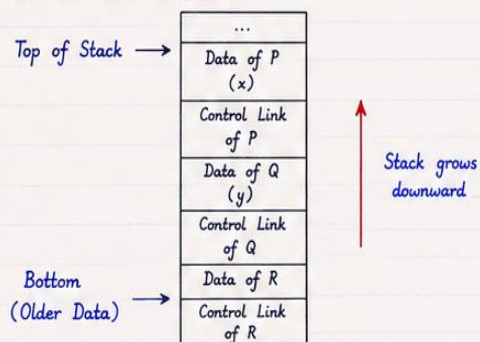
Consider the following nested procedures:

```
procedure P
  integer x ;
  procedure Q
    integer y ;
    procedure R
      x := x + 1 ; { access to nonlocal data x }
      y := y + 1 ; { access to nonlocal data y }
    end R
  end Q
end P
```

Activation (Call) Sequence :



3. Stack Organization



4. How Access to Nonlocal Data is Performed?

To access a nonlocal variable, the following information is required:

- Static (lexical) level of each procedure.
- Difference in levels between the current procedure and the procedure in which the nonlocal data is declared.
- Offset of the variable within its activation record.

Steps :

- Starting from the current activation record, follow static links (control links) up the stack.
- Move up (Level difference) times to reach the activation record of the procedure where the variable is declared.
- Use the offset within that record to access the variable.

5. Access Example in Above Program

When procedure R wants to access x and y :

Variable	Declared in	Level Difference from R	Access Method
x	P	2	Go up 2 static links and use offset of x
y	Q	1	Go up 1 static link and use offset of y

6. Important Points

- Static (lexical) links are used to access nonlocal data.
- This method supports nested procedures and block structure.
- It enables correct reference to variables in outer scopes.
- Efficient access to nonlocal data is essential for correctness and performance in compiled programs.

★ Access to nonlocal data on the stack is achieved by following static links to the proper activation record and using the known offset of the variable within that record.

5. Heap Management

Heap Management

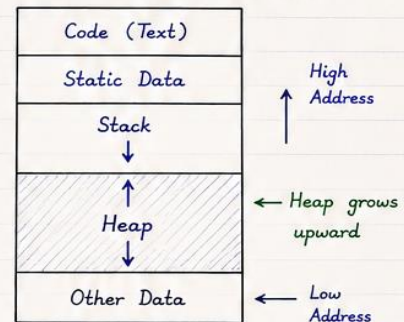
MODERN COMPILER DESIGN
(BE103701)

The heap is a region of memory used for dynamic memory allocation during program execution. It is managed by the run-time system.

1. What is Heap?

- Heap is used to allocate memory whose size or lifetime is not known at compile time.
- Memory on the heap remains allocated until it is explicitly freed.
- It supports dynamic data structures such as linked lists, trees, graphs, etc.
- Heap grows upward (towards higher addresses).

Run-time Memory Layout



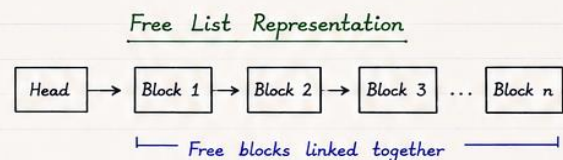
2. Heap Operations

Operation	Description	Example in C
Allocation (malloc / new)	Allocates a block of memory on heap of requested size. Returns pointer to the beginning of the block.	<pre>int *p = (int *) malloc (n * sizeof(int)); node *p = new node;</pre>
Deallocation (free / delete)	Frees the previously allocated memory block and makes it available for future use.	<pre>free(p); delete p;</pre>

3. Heap Management Techniques

a) Free List (Linked List of Free Blocks)

- All free blocks are linked together.
- When memory is requested, a block is taken from the free list.
- When memory is freed, the block is returned to the free list.



b) Placement Strategies

- First Fit - Allocate the first block that is large enough.
- Best Fit - Allocate the smallest block that is large enough.
- Worst Fit - Allocate the largest block.
- Next Fit - Continue search from where last search ended.

c) Coalescing

- When adjacent free blocks are available, they are merged into a single larger block.
- Coalescing reduces fragmentation and keeps the free list efficient.

Coalescing Example



4. Fragmentation

- Internal Fragmentation - Unused memory inside an allocated block.
- External Fragmentation - Free memory exists but not large enough in contiguous form.
- Coalescing helps to reduce external fragmentation.

5. Garbage Collection

- In some languages (e.g., Java), heap memory is automatically managed by garbage collector.
- In C/C++, programmer must explicitly free the memory.

★ Efficient heap management is essential for performance, memory utilization and to avoid memory leaks in compiled programs.